

From source code to a supported developer workflow

A Kubernetes platform design and local MVP
for the Testkube sample application.

01 / DEVELOP	02 / DELIVER	03 / OPERATE
Edit source. Get fast feedback.	Build, deploy and test through one workflow.	Diagnose failures. Recover deliberately.

Design stance

Start with a reliable path for one team. Make production responsibilities explicit. Add platform components when a demonstrated need justifies their operational cost.

Includes requirements analysis, architecture, ownership, delivery controls, tradeoffs and an incremental roadmap. The repository contains the executable MVP, developer guide and separate validation evidence.

1. Recommendation

Give the application team a supported path from source code to a tested Kubernetes deployment: a few familiar commands, application-oriented feedback, and documented recovery steps. Start with versioned containers, declarative manifests, and CI acceptance tests. Use the same versioned platform tooling for local development and shared CI/development delivery.

The platform is a product for developers. Its success is whether a new developer can deploy, understand a failure, and recover without a platform engineer translating Kubernetes objects for them. Kubernetes adoption itself is not the outcome.

This showcase implements **kind + Helm + Make**, backed by an installed platform CLI, for a React frontend, Express API, and in-cluster PostgreSQL. Developers can create a local kind cluster; GitHub-hosted CI deploys to an existing shared cluster. Feature pushes reset and test `app-ci`; main pushes promote already-tested images into `app-dev`. Manual production releases publish existing development images and release notes, then deploy to `app-prod` after GitHub environment approval. Managed services and Git reconciliation below are optional production directions, not requirements or implemented capabilities of this showcase.

2. Brief, assumptions, and questions

The case asks for deployment tooling, a strong developer experience, and a presentation or document with critical analysis. An MVP is encouraged. The proposal distinguishes case requirements, explicit assumptions, implemented capabilities, and production follow-up.

The following are explicit assumptions, not facts established by the case:

Assumption	Why it matters	What would change the design
One application team and one small service boundary	A repository workflow is a sufficient first interface	Many teams justify service templates and a catalog
GitHub is the collaboration and CI system	Reuses the repository's existing home	Use equivalent controls in the organization's CI
One shared showcase cluster plus developer-local clusters is acceptable	Local kind is inexpensive; CI reuses the existing cluster	Concurrent teams need isolated previews, quotas and expiry
Initial traffic is modest; no latency or uptime commitment exists	Resource values are starting estimates	Load tests and business SLOs determine capacity
Production data could matter even though this sample only reads a constant	Database recovery is a launch concern	Data classification determines encryption, retention and access
Managed services may be appropriate for a future real production system	Could reduce control-plane and database operations	This showcase uses PostgreSQL in Kubernetes; a real hosting choice needs separate agreement

Before production, confirm: who uses the app, who is on call, budget and hosting constraints, data classification, expected traffic, uptime expectations, and acceptable data loss and restoration time. Resolve these with product and engineering owners rather than inventing requirements to justify tools.

3. What the repository tells us

The application is a small three-tier system, but its original deployment contract was incomplete:

- The browser called `http://localhost:8080`; outside the developer laptop this points at the user's machine.
- The web Dockerfile used Vite's preview server. The platform preserves that developer-owned packaging.
- The API embedded database credentials in a connection URL and opened a client for each query. Failed database requests still returned HTTP 200.
- The API started listening when imported by tests and attached graceful shutdown handling to the Express app instead of the HTTP server.
- Existing Kubernetes examples deployed only the API. The ingress referenced a different Service name, and the image came from a temporary registry.
- The original API test checked only an HTTP success response; the browser test checked only a counter. Neither demonstrated that all three tiers worked together.

The MVP preserves the developer Dockerfiles, web application, root npm manifests and Docker Compose. One explicitly agreed API change reads database name, username and password from environment variables, with original defaults for direct local execution; an API unit test covers that behavior. Deployment tooling supplies Secret references, two local tunnels, existing-route probes and writable runtime volumes. Additional browser and outage acceptance tests live outside the application tree. Health status and shutdown improvements remain separate application work.

4. Architecture and developer journey

Local MVP

```
Developer -- make up --> dedicated kind cluster
      |               |
      +-- Docker builds --> local images loaded into kind

Browser: localhost:4173 --> web tunnel --> Vite preview + React
Browser: localhost:8080 --> API tunnel  --> Express --> PostgreSQL
                        |
                        1 GiB PVC
```

Both tunnels bind only to loopback. The original frontend hardcodes `localhost:8080` and the API enables CORS. This supports a laptop demonstration; it does not provide an environment-independent production frontend. Helm supplies an init container to copy the web project to a writable volume for Vite's generated configuration, while preserving the original Dockerfiles and startup commands.

A new developer installs the documented prerequisites, runs `make setup` to install the pinned CLI through `pipx`, then `make up` and `make open`. The application appears at one URL. `make deploy` rebuilds and deploys edits; `make check` verifies the complete request path; `make logs` and `make diagnose` explain operational failures. For rapid source edits, Vite and nodemon provide hot reload against a local development database.

The shared application chart in the tagged `platform-tools` package contains one Deployment and Service. It is installed independently as releases `api` and `web`, using per-service values in `deploy/`. PostgreSQL is release `db`, provisioned by a separately version- and checksum-pinned upstream chart; it is not an application chart dependency. Schema-validated application values configure image, replicas, port, probes, resources and runtime storage. Helm manages upgrades and revision history; the wrapper builds and loads local images and selects their references without modifying tracked files. Every Kubernetes command uses a dedicated kubeconfig and explicit context. The script does not depend on whichever cluster the developer happened to select earlier.

Shared CI and development delivery (implemented)

```
Feature push -> reset app-ci releases and database data
              -> build api/web -> public GHCR ci-<run> and ci-latest
              -> install db, api, web -> tests -> mark ci-passed
Merge to main -> verify tested images and merged source tree
              -> add dev-latest -> upgrade app-dev -> HTTP checks
Manual release -> publish prod-<UTC> + latest and PR-delta notes
              -> app-prod approval -> deploy recorded digests
```

The existing cluster uses context `kind-testkube-samples`. GitHub environments `app-ci` and `app-dev` supply their KUBECONFIG and DB_NAME, DB_USER and DB_PASSWORD secrets. Hosted runners must reach the API and publish public GHCR images. CI never creates or deletes the cluster. Each namespace owns separate `api`, `web` and `db` releases; development data persists, while every feature run deletes CI releases, the database PVC and its Secret.

One repository-wide concurrency group serializes CI and promotion. GitHub may replace a pending run with a newer pending run; a required canceled check must be rerun. Images deploy as tag-plus-digest references. The `ci-passed` marker is written only after acceptance succeeds. Promotion checks that both latest images match that marker, share a CI build, and have the same source tree as main; it never rebuilds. Another branch moving `ci-latest` can block promotion, so this shared namespace is a deliberate showcase limitation. Require up-to-date branches and the Platform gate before merging. Main deployment occurs after merge; failure cannot undo the merge.

The production workflow runs only from main after successful development delivery. Its first job has no environment: it validates dev-latest against the post-deployment dev-passed marker, records image digests and the source commit in a release asset, publishes prod-UTCtimestamp/latest image tags, and creates a GitHub Release listing merged PRs since the previous published production release. Its second job selects app-prod, waits for environment protection rules, verifies the recorded manifest and deploys exact digests with that environment's database/cluster secrets. Artifacts exist before approval and may remain undeployed. Retries reuse the run's timestamp and snapshot; newer production records block older retries. The shared concurrency lock is held during approval, so CI and development wait. See PRODUCTION.md for operation and first-release baseline details.

`make local-tests` is developer-only and retains the local kind flow. The disruptive `make resilience` drill is optional for the showcase and included in that local suite, but excluded from required delivery CI.

Future production hardening (not implemented)

```
Feature change -> build once -> registry image digests + CI checks
              -> review -> merge -> select tested image digests
              -> staging environment PR -> Git reconciler -> staging
              -> smoke checks + approval -> production environment PR
              -> Git reconciler -> managed Kubernetes
```

```
Users -> TLS gateway -> web -> API -> managed PostgreSQL
              |           |
              logs/metrics backups + tested restore
```

An environment repository records reviewed image digests and configuration. CI publishes artifacts but does not hold a long-lived production kubeconfig. A reconciler applies the reviewed desired state. Production rollback reverts the environment commit, followed by verification; emergency changes must be reconciled back into Git. Database migration compatibility remains a separate responsibility.

5. Critical requirements and tradeoffs

Developer experience and ownership

Priority: essential. The team is unfamiliar with Kubernetes, so exposing a large chart configuration or asking developers to learn cluster administration would transfer platform work to them. Commands should describe tasks, fail clearly, and link to a runbook. The underlying YAML remains readable as an escape hatch.

Make is broadly available and sufficient for this service. Its limitations are shell portability, installation friction, and less structured validation. The versioned CLI is already installed through `make setup`; a dev container could further reduce prerequisite setup if onboarding evidence justifies it. A portal comes after a useful workflow; otherwise it merely wraps unreliable steps in a UI.

The application team owns application behavior, dependencies, schema changes, and service health. The platform team owns templates, cluster lifecycle, delivery controls, baseline security, and shared telemetry. Both own actionable alerts and incident practice. Platform changes need versioning and a migration path, not silent changes to every team's templates.

Versioned platform ownership

Reusable scripts, the shared Helm chart, locked chart-test dependencies and the reusable CI workflow are owned in `taskovskig/platform-tools`. This repository keeps a thin Makefile, declarative service configuration, per-service values, application-specific acceptance tests and interview documentation. Bootstrap source, its tests and the PDF renderer live in `platform-tools`; no bootstrap source is copied into application repositories. Both application Dockerfiles remain developer-owned.

`platform.lock.json` selects a semantic-version Git tag, manifest checksum and CLI API compatibility version. `make setup` installs the CLI from the reviewed tag; CI installs it in a runner-local virtual environment. The installed CLI verifies the manifest and every packaged file before execution, caches by checksum, and fails on missing or changed tag content. CI references the same tag explicitly. Platform upgrades therefore become application review decisions rather than silent changes. Published tags must remain immutable. Local checkout overrides support pre-release testing and are prohibited in CI.

Separating repositories introduces a release-order dependency: publish the platform tag before the consumer PR can pass hosted CI. It also adds bootstrap ownership and compatibility work. A shared chart is bundled with its runtime release to avoid independently selecting incompatible versions; environment values and application behavior remain in the application repository.

Correct deployment and feedback

Priority: essential. A green pod is insufficient evidence that the application works. The MVP verifies HTML delivery, API output, a real SQL query, and browser interactions. The same deployment script runs locally and in CI, reducing a class of environment differences.

Helm provides a versioned application package, a constrained values interface and standard release history and rollback. The chart keeps health checks and security defaults in templates rather than exposing every Kubernetes field. Template maintenance and one additional client tool are costs we accept for consistent packaging. `helm template` preserves inspectability, and schema plus rendered-resource tests catch configuration mistakes. The upstream repository's separate chart packages Testkube workflows; it does not supply this application deployment. `kind` is a convenient disposable integration environment, but does not reproduce a cloud load balancer, multi-zone storage, IAM, or production failure modes.

Local application images get a unique local release tag. Shared CI publishes `ci-<GitHub run number>` and `ci-latest`; reruns can reuse a numbered tag, so deployment also pins the resolved digest. Development uses `dev-latest` with a digest. Dependencies use the committed npm lockfile; the `kind` node uses a digest. The developer Dockerfiles use the floating `node:lts` base and retain development dependencies; PostgreSQL uses

a version-line tag, so builds are **not fully immutable**. Production requires digest pinning, automated update PRs, retained registry artifacts, image scanning and provenance. The repository's old dependencies also need a deliberate upgrade and vulnerability triage effort; packaging them in a container does not resolve that risk.

Availability and safe releases

Priority: essential, with production targets to agree. API startup, readiness and liveness all use the existing `/hello` route. They prove process responsiveness only. The original API stays Ready during a database outage and returns HTTP 200 with a failure message. Smoke tests therefore assert the exact SQL response, and the outage drill records this limitation instead of claiming database-aware readiness.

Stateless rollouts request zero unavailable replicas with one surge pod. This requires spare capacity and does not promise zero downtime. The original shutdown handler is attached to the Express app rather than its HTTP server; reliable connection draining is not established. Production requires developer-owned fixes for health, status codes and shutdown, plus traffic-under-rollout testing.

The local rollback command selects `SERVICE=api` or `SERVICE=web` and restores that release's prior Helm revision, including its manifests, values and image reference. The other application and database releases are untouched. It needs retained images. Deploying both applications is sequential, not atomic. Database upgrades are an explicit separate operation. PostgreSQL data, schema, external Secrets and volume resizing are not rolled back. Production needs backward-compatible API changes and expand/contract migrations, plus an explicit owner for deciding between rollback and a forward fix.

Data durability and recovery

Priority: essential before real data. A PostgreSQL StatefulSet and PVC in each deployed namespace demonstrate persistence across pod replacement. They do not provide high availability, off-machine backups, or recovery after cluster deletion. The sample's SQL query reads a constant; a dedicated persistence drill is necessary to prove storage behavior.

For production, choose managed PostgreSQL with automated backups, point-in-time recovery, encryption, monitoring, and a tested restoration procedure. Kubernetes can run databases, but the operational burden needs a reason. Separate the migration role from the application's least-privileged database role; the showcase's application user is also the initialization superuser in local and shared namespaces, a simplification that requires revisiting before real production use.

No recovery target is implied by the brief. A proposed discussion starting point is RPO of 15 minutes and RTO of one hour. Product and operations must accept or replace those targets, select a service tier, and prove them through restoration drills before promising them.

Security and environment boundaries

Priority: essential for shared environments. The MVP runs containers as non-root, drops Linux capabilities, uses read-only root filesystems with specific writable volumes, disables service-account token mounts, and enforces the restricted pod-security profile. Both local tunnels bind to loopback. Local deployments read database defaults from `platform.json`. Shared deployments require `DB_NAME`, `DB_USER` and `DB_PASSWORD` from the selected GitHub environment and create a namespace-local database Secret. PostgreSQL and the API consume matching keys through Secret references; passwords are not Helm values. Direct local API execution still has sample defaults. Separate environment credentials reduce accidental cross-environment access but do not establish strong isolation. Local platform state is excluded from the allowlisted Docker build context.

These controls do not establish tenant isolation. The default kind network is not presented as a policy enforcement solution. A shared platform needs a CNI that enforces NetworkPolicy, default-deny rules with tested DNS/web/API/database allowances, scoped RBAC, quotas, workload identity, and external secret management.

Use TLS and an agreed authentication model for public access. Production and development should have separate access and failure boundaries; namespaces alone are insufficient for hostile tenants.

Secrets in Kubernetes are not protected merely because they are encoded. Restrict reads, configure encryption at rest, audit access, and rotate credentials with the database. Updating a Secret without changing the actual database password can break the application. Retained PVCs must never be silently paired with a regenerated password. The deployment stops when an existing Secret differs from the requested name or credentials, including a legacy Secret missing the database-name key. It does not rotate database credentials. CI initializes fresh data; development initializes on first deployment and retains it thereafter.

Operability and cost

Priority: basic visibility now, service-level monitoring before production. The MVP provides stdout logs, deployment events, existing-route probes, rollout timeouts and a troubleshooting guide. It does not claim centralized logs, metrics dashboards, tracing, or alerting.

In a shared environment, measure request rate, error rate and duration; database connection pressure; pod restarts; and saturation. Associate release identity with telemetry. Agree an SLO and alert on sustained user impact and error-budget burn, with links to a runbook. Add tracing when a multi-service request path makes it useful, not because Kubernetes requires it.

Resource requests and limits are estimates, not capacity measurements. Measure under representative load, then tune. Proposed production web/API replicas should span failure domains; disruption budgets and autoscaling must follow actual availability and traffic requirements. Database capacity and connection limits can be the scaling constraint. Local kind keeps the demo inexpensive; cloud spend should be estimated from the agreed region, node footprint, database tier, backups and telemetry retention.

6. Delivery, acceptance, and scope

Capability	MVP status	Production follow-up
Independent application releases	Tagged platform chart; separate api and web values	Managed cluster and environment overlays
Developer commands and runbook	Implemented	Onboarding measurement and template versioning
Acceptance environments	Local kind; serialized shared app-ci; digest-pinned deployments	Immutable base digests and artifact provenance
CI quality gate	Feature tests and aggregate Platform gate; main promotion checks	Verify branch protection and add production approvals
Health and deployment feedback	Probes, rollout waits, smoke and browser tests	SLOs, telemetry and alerting
Database persistence	Separate db release and PVC per deployed namespace	Managed database, backup/restore evidence
Credential handling	Environment secrets shared by API and PostgreSQL; local defaults	External store, rotation and scoped database role
Release recovery	Helm revisions; tested-image promotion and approved app-prod releases	Git-based promotion, retention, migration policy
Security baseline	Non-root restricted workloads	RBAC, enforced network policy, TLS, identity

The delivery workflow runs on branch pushes, with repository contents read and packages write permissions. Feature runs `reset app-ci`, build and publish images, run chart validation, API unit tests, frontend build, deployment HTTP checks, Helm release isolation/rollback tests and browser tests. Diagnostics run on failure; runner credentials are removed, but cluster workloads remain for inspection. Helm tests upgrade and roll back api/web, so successful runs normally leave those releases at revision 3 and db at revision 1.

Main runs validate charts, verify image provenance against the tested CI images and merged source tree, retag those images, upgrade app-dev and check HTTP contracts including the database-backed endpoint. Unit, browser and Helm mutation tests are not repeated on main. Resilience fault injection is not required CI. The separate manual workflow publishes a production snapshot without an environment, then an app-prod job waits for approval and deploys its recorded digests. Hosted CI success is evidence for the tested commit, not a guarantee of future network availability or production readiness.

See `VALIDATION.md` for observed results and remaining limitations. Evidence should always distinguish a manifest that renders, a container that builds, a pod that is ready, and a browser journey that works.

7. Incremental platform roadmap

- 1. Prove the local path.** Observe a new developer onboarding and improve instructions. Measure whether initial deployment takes under 15 minutes after prerequisites and cached downloads; this is a target, not a measured result.

2. **Agree the application contract.** Configurable database credentials and name are implemented and tested. Remaining developer-owned changes are relative or configurable API URLs, production web serving, correct failure status, database-aware readiness and graceful shutdown.
3. **Harden the implemented shared environments.** app-ci and app-dev already use the existing kind cluster, GHCR and in-cluster PostgreSQL. Add access and capacity evidence, image scanning, TLS, migration checks and restoration drills as needed. A future production hosting and reconciliation model requires a separate decision.
4. **Meet production readiness criteria.** Agree SLO/RPO/RTO and ownership; test rollback, backup recovery, access restrictions, capacity and meaningful failure scenarios. Measure the actual recovery times.
5. **Scale the product based on demand.** Add preview environments with quotas and expiry, reusable service templates, and a service catalog when more teams need them. Consider Testkube if centrally scheduled or cross-environment test orchestration becomes a real need; the repository name alone does not justify installing it.

Track onboarding time, deployment lead time, recovery time, change failure rate and support requests. Use developer feedback to prioritize improvements.

8. Interview discussion guide

Present the problem and assumptions first, then show the running app and one deployment. Explain why application ownership, honest readiness signals and data recovery matter more than adding a catalog. Optionally demonstrate the local database outage/recovery drill, and identify which production responsibilities the demo intentionally leaves open.

Discussion prompts: Kubernetes is a case constraint; a managed application runtime may suit a real small service. Helm supplies packaging and revision history, while Make and the CLI simplify developer commands. The upstream chart contains Testkube workflows, not application workloads. In-cluster PostgreSQL keeps this showcase self-contained; production hosting requires a separate recovery decision. Next steps should address shared access and restoration evidence before adding a service mesh or catalog.

References

- [kind quick start](#) and [v0.33.0 release](#): cluster setup, local image loading and the selected node image.
- [Kubernetes probe documentation](#): liveness, readiness and startup semantics.
- [Helm charts](#), [upgrade](#) and [rollback](#): packaging, values and release lifecycle.
- [Upstream workflow chart](#): Testkube examples, separate from the shared application chart implemented here.
- [PostgreSQL container documentation](#): initialization variables and persistent data directory behavior.
- [groundhog2k PostgreSQL chart](#): pinned at 1.6.8; independent database upgrades for local and shared deployments.